O método de Hannan-Rissanen

primeiro a ideia, depois o seu ARIMA, linha por linha

Material de estudo para os autores do TG (não faz parte do texto)

versão 2, 2 de novembro de 2026

Em uma página

O problema. O ARIMA usa duas coisas para prever o retorno de amanhã: os retornos passados (parte AR) e os *erros* passados, as surpresas que o modelo não previu (parte MA). Os retornos estão nos dados. Os erros não: eles dependem dos próprios coeficientes que se quer estimar.
A ideia central. O erro de hoje pode ser escrito como uma soma ponderada dos retornos passados. Então um modelo que só usa retornos passados (um AR comprido) consegue *estimar* os erros, sem precisar deles.
O método, em duas regressões. Estágio 1: ajusta-se um AR comprido e guarda-se o que ele não explicou, que é a estimativa dos erros. Estágio 2: esses erros estimados viram uma coluna a mais numa segunda regressão, que dá os coeficientes do ARIMA. As duas regressões são mínimos quadrados (MQO), uma conta fechada, sem tentativa e erro.
No seu robô. É a função `CArima::HannanRissanen`, chamada todo pregão para o modelo em uso e 25 vezes a cada 21 pregões, na escolha das ordens. Refazendo fora do robô dois pregões reais, os coeficientes e a previsão saíram idênticos aos que o robô registrou.
Como ler. A Parte I explica a ideia sem falar do robô. A Parte II segue o caminho do dado dentro do código, do fechamento da PETR4 até a previsão, com os trechos de código na mesma numeração dos Apêndices A e B do TG. A Parte III mostra quão bem o método estima e onde ele difere da literatura.

Sumário

I A ideia, sem o robô **2**

1 A única ferramenta: regressão por mínimos quadrados **2**

2 Parte AR: o passado vira coluna **4**

3 Parte MA: a coluna que falta **4**

4 A ideia central: o erro de hoje é uma soma de retornos passados **5**

5 O método completo, em duas regressões **5**

II O método dentro do seu ARIMA **6**

6 O caminho do dado no robô **6**

6.1 `Ajustar`: a função que organiza tudo 8

7 Primeiro, o atalho: modelo sem termo MA **9**

8 Estágio 1 no código: o AR comprido **10**

8.1 Quantos dias para trás 10

8.2 O AR comprido 10

8.3 O que o estágio 1 produziu no pregão real 11

9 Estágio 2 no código: a regressão que dá c e θ **12**

10 Depois dos dois estágios: os resíduos finais **13**

11 A previsão de amanhã **14**

12 Na escolha das ordens: 25 modelos de uma vez **16**

13 Um pregão real, inteiro: 13/05/2021 **17**

14 Quanto ele trabalha no robô **18**

III Quão bem ele estima **18**

15 Mil simulações **18**

16 O seu código comparado ao método da literatura **20**

16.1 Um ponto para pensar: a ordem do AR comprido numa janela pequena 20

17 Perguntas que a banca pode fazer **22**

Parte I

A ideia, sem o robô

1 A única ferramenta: regressão por mínimos quadrados

Tudo no Hannan-Rissanen é feito com uma ferramenta só, então vale começar por ela.

Uma **regressão** é uma tabela. Cada linha é uma observação (aqui, um dia). Há uma coluna que se quer explicar, chamada y, e algumas colunas que explicam, $x_1, x_2, \ldots$. A regressão procura os pesos $b_1, b_2, \ldots$ tais que $b_1 x_1 + b_2 x_2 + \ldots$ fique o mais perto possível de y em todas as linhas ao mesmo tempo.

Mínimos quadrados ordinários (MQO) é a regra para decidir o que é "mais perto": os pesos escolhidos

são os que tornam mínima a soma dos erros ao quadrado,

$$\text{escolher } b \text{ que minimiza } \sum_{\text{linhas}} \left(y - b_1 x_1 - b_2 x_2 - \ldots\right)^2.$$

A vantagem do MQO é que a resposta sai de uma conta fechada. Escrevendo a tabela das colunas explicativas como uma matriz X (uma linha por dia, uma coluna por variável), os pesos são a solução do sistema linear

$$(X^\top X)\, b = X^\top y.$$

$X^\top X$ é uma tabela pequena (tantas linhas e colunas quantas forem as variáveis) com as somas dos produtos entre cada par de colunas. $X^\top y$ tem a soma dos produtos de cada coluna com y. Montadas as duas, resolve-se o sistema e pronto: não há chute inicial, número de iterações nem taxa de aprendizado.

No projeto, essa conta mora num lugar só, a função `MQO` de `Numerico.mqh`:

robos/Include/Numerico.mqh

```
30    for(int i = 0; i < parametros; i++)
31      {
32       double soma_xy = 0.0;
33       for(int t = 0; t < observacoes; t++)
34          soma_xy += X.Get(t, i) * y[t];
35       Xty[i] = soma_xy;
36
37       for(int j = 0; j <= i; j++)
38         {
39          double soma = 0.0;
40          for(int t = 0; t < observacoes; t++)
41             soma += X.Get(t, i) * X.Get(t, j);
42
43          XtX.Set(i, j, soma);
44          XtX.Set(j, i, soma);
45          XtX_para_inversa.Set(i, j, soma);
46          XtX_para_inversa.Set(j, i, soma);
47         }
48      }
49
50    int situacao = 0;
51    CDenseSolverReportShell relatorio_solver;
52    ArrayResize(coeficientes, parametros);
53    ArrayInitialize(coeficientes, 0.0);
54
55    CAlglib::RMatrixSolve(XtX, parametros, Xty, situacao, relatorio_solver, coeficientes);
56    if(situacao <= 0 || ArraySize(coeficientes) < parametros)
57       return(false);
```

Lendo o código

Linhas 30–48: montam $X^\top y$ (variável `Xty`) e $X^\top X$ (`XtX`) somando, linha a linha (t = dia), os produtos das colunas. Como $X^\top X$ é simétrica, só metade é calculada e a outra é copiada (linhas 43–44).
Linha 55: a ALGLIB resolve o sistema $(X^\top X)\, b = X^\top y$. O resultado vai para `coeficientes`.
Linha 56: se a matriz for singular (duas colunas iguais, por exemplo), a função devolve falso e o modelo não é ajustado naquele dia.

O ponto que importa para o resto do relatório

O MQO só funciona se **todas as colunas da tabela forem números conhecidos**. Se uma coluna não pode ser calculada a partir dos dados, não há tabela, e não há regressão.

2 Parte AR: o passado vira coluna

O ARIMA do robô, depois de tirar o log do preço e diferenciar uma vez ($d = 1$), trabalha com o log-retorno z_t de cada dia. Um modelo **AR(p)** diz que o retorno de hoje é uma constante mais uma combinação dos p retornos anteriores, mais um erro:

$$z_t = c + \phi_1 z_{t-1} + \phi_2 z_{t-2} + \cdots + \phi_p z_{t-p} + \varepsilon_t .$$

Isso já é uma regressão. A coluna y é o retorno do dia, e as colunas x são uma coluna de 1 (para a constante) e os retornos dos dias anteriores, todos conhecidos. Veja as primeiras linhas montadas pelo robô num pregão real (que será detalhado na seção 13), com retornos em %:

dia t	z_t	1	z_{t-1}	z_{t-2}	z_{t-3}		z_{t-29}
07/07/2020	−1,252	1	2,383	−0,363	1,599	⋯	0,971
08/07/2020	1,872	1	−1,252	2,383	−0,363	⋯	1,313
09/07/2020	−2,277	1	1,872	−1,252	2,383	⋯	−0,806
10/07/2020	1,657	1	−2,277	1,872	−1,252	⋯	2,842
			⋮				

Repare na diagonal: o z_t de uma linha reaparece como z_{t-1} na linha de baixo, e como z_{t-2} na seguinte. É a mesma série, deslocada um dia em cada coluna. Montada a tabela, o MQO da seção anterior dá c e os ϕ.

3 Parte MA: a coluna que falta

O ε_t da equação acima é o **erro** do dia: a parte do retorno que o modelo não conseguiu prever, a surpresa. Um modelo **MA(q)** (médias móveis dos erros) usa as surpresas dos dias anteriores para prever hoje:

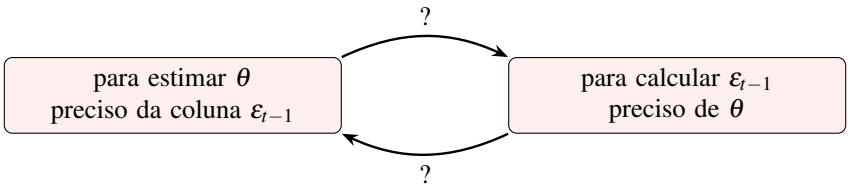
$$z_t = c + \theta_1 \varepsilon_{t-1} + \cdots + \theta_q \varepsilon_{t-q} + \varepsilon_t .$$

A intuição: se ontem o mercado surpreendeu para baixo, isso pode dizer algo sobre hoje. Um θ negativo, por exemplo, transforma uma surpresa negativa de ontem num empurrão para cima hoje, uma pequena correção.

Juntando as duas partes tem-se o **ARMA(p, q)**, que é o que o robô estima (o "I" do ARIMA é só a diferença feita antes):

$$z_t \;=\; c \;+\; \underbrace{\phi_1 z_{t-1} + \cdots + \phi_p z_{t-p}}_{\text{parte AR: retornos passados}} \;+\; \underbrace{\theta_1 \varepsilon_{t-1} + \cdots + \theta_q \varepsilon_{t-q}}_{\text{parte MA: erros passados}} \;+\; \varepsilon_t .$$

Para estimar isso por MQO, a tabela precisaria de uma coluna ε_{t-1}. Mas o erro de ontem não está em lugar nenhum dos dados. Ele é a diferença entre o retorno de ontem e o que o modelo previa ontem, e o que o modelo previa depende de θ. É uma volta em círculo:



Há duas formas de sair do círculo. A **máxima verossimilhança** chuta valores de θ, calcula os erros que eles implicam, mede o quão plausíveis eles são e repete até não melhorar mais. Funciona, mas é iterativa:

precisa de um ponto de partida, de um critério de parada, e pode não convergir. O **Hannan-Rissanen** sai do círculo de outro jeito: estima os erros *antes*, por um caminho que não passa por θ. O caminho é a próxima seção.

E se simplesmente ignorar a parte MA?

Numa simulação com $\phi = 0{,}6$ e $\theta = 0{,}4$ verdadeiros (seção 15), um AR(1) que ignora o MA estimou, em média, $\phi = 0{,}746$ em vez de 0,6. O efeito do MA não some quando é ignorado: ele vai parar dentro do coeficiente AR e o distorce.

4 A ideia central: o erro de hoje é uma soma de retornos passados

Este é o passo que destrava tudo, e vale fazê-lo devagar no caso mais simples, um MA(1):

$$z_t = c + \varepsilon_t + \theta\,\varepsilon_{t-1}.$$

Passo 1. Isola-se o erro de hoje:

$$\varepsilon_t = (z_t - c) - \theta\,\varepsilon_{t-1}.$$

Passo 2. A mesma equação vale para ontem: $\varepsilon_{t-1} = (z_{t-1} - c) - \theta\,\varepsilon_{t-2}$. Substituindo:

$$\varepsilon_t = (z_t - c) - \theta\,(z_{t-1} - c) + \theta^2\,\varepsilon_{t-2}.$$

Passo 3. Repetindo para anteontem, e assim por diante:

$$\varepsilon_t = (z_t - c) - \theta\,(z_{t-1} - c) + \theta^2\,(z_{t-2} - c) - \theta^3\,(z_{t-3} - c) + \ldots$$

O que apareceu. Do lado direito só há retornos, que são conhecidos. O erro, que não se via, é uma soma ponderada dos retornos passados. Os pesos são potências de θ: se $|\theta| < 1$ (a condição chamada *invertibilidade*), eles encolhem a cada dia para trás, e a soma converge.

Virando a conta ao contrário. Isolando z_t nessa última linha, o MA(1) vira

$$z_t = \text{constante} + \pi_1 z_{t-1} + \pi_2 z_{t-2} + \pi_3 z_{t-3} + \cdots + \varepsilon_t, \qquad \pi_j = -(-\theta)^j.$$

Isso é um AR, só que com infinitos termos. Em palavras: **todo MA invertível é um AR comprido**. O mesmo vale para o ARMA(p,q) completo; no ARMA(1,1) os pesos ficam $\pi_j = (\phi + \theta)(-\theta)^{j-1}$.

Com o número do robô. No pregão real da seção 13, o robô estimou $\theta = -0{,}0929$. Os pesos do AR equivalente seriam $\pi_1 = -0{,}0929$, $\pi_2 = -0{,}0086$ e $\pi_3 = -0{,}0008$: a partir do terceiro dia para trás, o peso já é menor que um milésimo. Um AR que vai algumas dezenas de dias para trás carrega praticamente tudo.

A consequência. Um AR comprido, mas finito, aproxima o ARMA. E um AR se estima por MQO, porque todas as colunas são retornos. Então dá para ajustar um AR comprido, olhar o que ele não explicou em cada dia, e usar isso como estimativa do erro, sem nunca ter precisado de θ.

A Figura 1 mostra isso numa série simulada de um ARMA(1,1) com $\phi = 0{,}6$ e $\theta = 0{,}4$, do mesmo tamanho da janela do robô (239 retornos). As barras azuis são os pesos verdadeiros do AR infinito. Os pontos laranja são o que um AR(29) ajustado por MQO estimou a partir dos dados.

5 O método completo, em duas regressões

Com a ideia da seção anterior, o método de Hannan e Rissanen tem dois estágios, e cada um é um MQO.

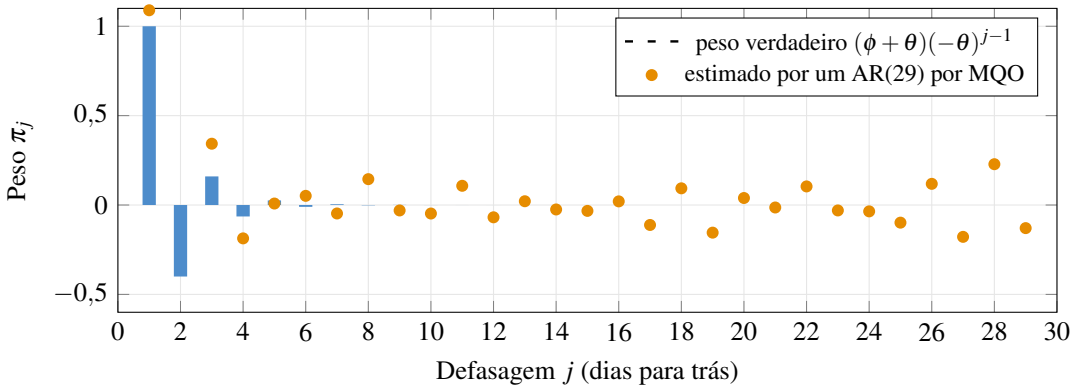


Figura 1: Série simulada: o AR comprido recupera os primeiros pesos, que são os que importam. Depois da quinta defasagem os pesos verdadeiros são quase zero, e o que o AR estima ali é ruído.

Estágio 1, estimar os erros. Ajusta-se um AR de ordem m grande à série. O resíduo de cada dia,

$$\hat{\varepsilon}_t = z_t - \left(\hat{a}_0 + \hat{a}_1 z_{t-1} + \cdots + \hat{a}_m z_{t-m}\right),$$

é a estimativa do erro daquele dia. Os coeficientes $\hat{a}$ do AR comprido são descartados: só os resíduos interessam.

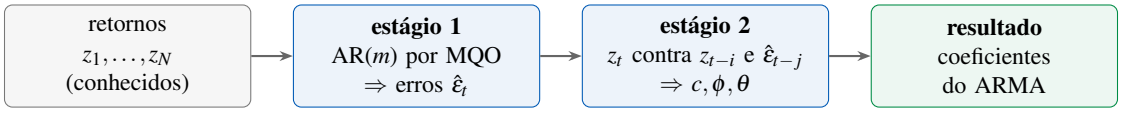
Estágio 2, estimar o modelo. Agora existe a coluna que faltava. Regride-se z_t contra a constante, os p retornos passados e os q erros estimados passados:

$$z_t = c + \phi_1 z_{t-1} + \cdots + \phi_p z_{t-p} + \theta_1 \hat{\varepsilon}_{t-1} + \cdots + \theta_q \hat{\varepsilon}_{t-q} + \text{resto}.$$

Os pesos dessa regressão são o resultado: c, os ϕ e os θ.

Uma analogia. Imagine que se quer medir o efeito de uma variável que não se observa diretamente. Primeiro, constrói-se uma estimativa dela a partir do que se observa. Depois, usa-se essa estimativa como se fosse o dado. A estimativa não é perfeita, e por isso o resultado é uma aproximação, mas é uma aproximação que se calcula com duas contas fechadas.

O que a literatura acrescenta. Gómez e Maravall (1998, p. 28–29) descrevem um terceiro passo, uma correção do viés dos coeficientes do estágio 2 quando há termo MA. O robô *não* faz esse terceiro passo (seção 16).



Fim da Parte I. Daqui em diante, o mesmo método dentro do código do robô.

Parte II

O método dentro do seu ARIMA

6 O caminho do dado no robô

Antes de entrar na função `HannanRissanen`, vale ver de onde ela é chamada. A cada pregão novo, o `OnTick` do EA faz isto (Apêndice A):

tg/codigo/ARIMA_GARCH_EA.mq5 (Apêndice A)

```
60 void OnTick()
61   {
62    datetime barra = (datetime)SeriesInfoInteger(_Symbol, _Period, SERIES_LASTBAR_DATE);
63    if(barra == g_ultima_barra)
64       return;
65    g_ultima_barra = barra;
66
67    ReselecionarOrdensSeVencidas();
68    if(!g_ordens_definidas)
69       return;
70
71    double serie[];
72    if(!CarregarSerie(Janela_Estimacao, serie))
73       return;
74
75    if(!g_arima.Ajustar(serie))
76       return;
77
78    double previsao = PrevisaoDoRetorno(serie);
79    if(!MathIsValidNumber(previsao))
80       return;
```

Lendo o código

Linha 67: a cada 21 pregões, reescolhe as ordens (p, d, q) (seção 12). É ali que o Hannan-Rissanen roda mais vezes.
Linha 72: carrega os últimos fechamentos (240 na configuração do TG) e tira o log de cada um.
Linha 75: `Ajustar` estima o ARIMA com as ordens em uso. É dentro dele que o Hannan-Rissanen roda.
Linha 78: a previsão do retorno de amanhã, que dá a direção da operação.

A Figura 2 resume o caminho inteiro. As caixas azuis são as funções desta parte do relatório.

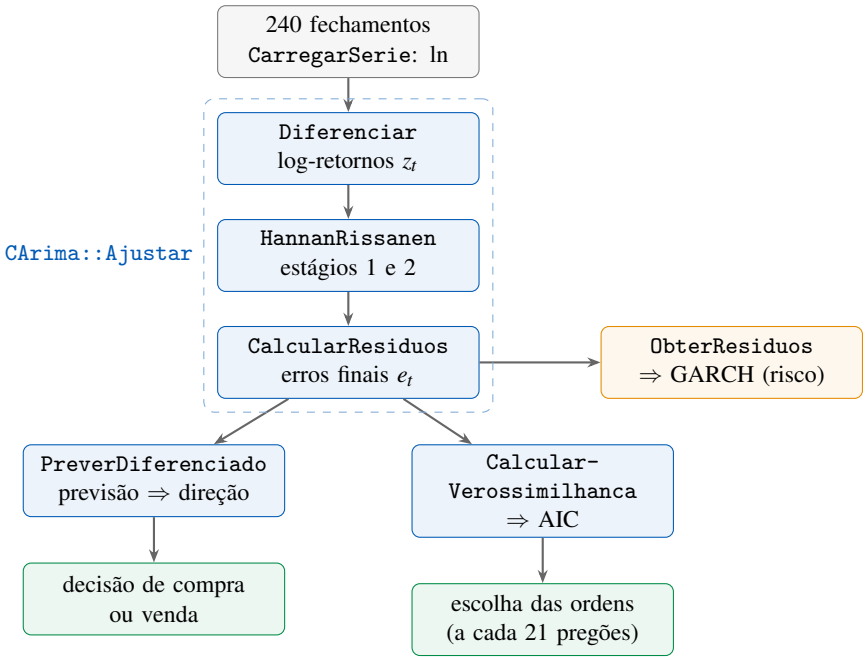


Figura 2: O caminho do fechamento até a decisão. O Hannan-Rissanen é a terceira caixa, mas os resíduos que ele permite calcular alimentam três lugares: a previsão, a escolha das ordens e o GARCH.

6.1 `Ajustar`: a função que organiza tudo

tg/codigo/Arima.mqh (Apêndice B)

```
267 bool CArima::Ajustar(const double &serie[])
268   {
269    m_ajustado = false;
270
271    int n = ArraySize(serie);
272    int minimo = m_p + m_q + m_d + 30;
273    if(n < minimo)
274       return(false);
275
276    ArrayResize(m_ultimos_niveis, MathMin(n, 3));
277    for(int i = 0; i < ArraySize(m_ultimos_niveis); i++)
278       m_ultimos_niveis[i] = serie[n - ArraySize(m_ultimos_niveis) + i];
279
280    Diferenciar(serie, m_serie_diferenciada, m_d);
281    if(ArraySize(m_serie_diferenciada) < m_p + m_q + 20)
282       return(false);
283
284    ArrayResize(m_phi, m_p);
285    ArrayResize(m_theta, m_q);
286    ArrayInitialize(m_phi, 0.0);
287    ArrayInitialize(m_theta, 0.0);
288    m_constante = 0.0;
289
290    if(!HannanRissanen())
291       return(false);
292
293    for(int i = 0; i < m_p; i++)
294       if(!MathIsValidNumber(m_phi[i]))
295          return(false);
296    for(int j = 0; j < m_q; j++)
297       if(!MathIsValidNumber(m_theta[j]))
298          return(false);
299
300    CalcularResiduos();
301    CalcularVerossimilhanca();
302
303    if(m_log_verossimilhanca == -DBL_MAX)
304       return(false);
305
306    m_ajustado = true;
307    return(true);
308   }
```

Lendo o código

Linhas 276–278: guarda os três últimos log-preços, para depois transformar a previsão do retorno em previsão do nível, se for preciso.
Linha 280: diferencia a série d vezes. Com $d = 1$, o log-preço vira log-retorno: $z_t = \ln P_t - \ln P_{t-1}$.
Linha 290: o Hannan-Rissanen. Se falhar (série curta, matriz singular), o modelo não é ajustado naquele dia.
Linhas 293–298: recusa coeficientes inválidos (`NaN` ou infinito).
Linhas 300–301: com os coeficientes prontos, recalcula os resíduos e a verossimilhança (seções 10 e 12).

A diferença é feita assim, com o índice 0 sendo o dia mais antigo:

tg/codigo/Arima.mqh

```
77 void CArima::Diferenciar(const double &origem[], double &destino[], const int ordem)
78   {
79    ArrayResize(destino, ArraySize(origem));
80    ArrayCopy(destino, origem);
81 
82    for(int vez = 0; vez < ordem; vez++)
83      {
84       int tamanho = ArraySize(destino);
85       if(tamanho < 2)
86          break;
87       for(int i = 0; i < tamanho - 1; i++)
88          destino[i] = destino[i + 1] - destino[i];
89       ArrayResize(destino, tamanho - 1);
90      }
91   }
```

Cada passada do laço da linha 82 troca a série pela diferença entre dias vizinhos (linha 88) e a encurta em um (linha 89). Com 240 log-preços e $d = 1$, saem $N = 239$ log-retornos. É sobre eles que tudo o mais trabalha.

7 Primeiro, o atalho: modelo sem termo MA

A função `HannanRissanen` começa verificando se há termo MA. Se não houver, o círculo da seção 3 não existe, e uma regressão comum resolve:

tg/codigo/Arima.mqh

```
131 bool CArima::HannanRissanen(void)
132   {
133    int tamanho = ArraySize(m_serie_diferenciada);
134 
135    if(m_q == 0)
136      {
137       if(m_p == 0)
138         {
139          m_constante = m_com_constante ? Media(m_serie_diferenciada) : 0.0;
140          return(true);
141         }
142 
143       int observacoes = tamanho - m_p;
144       int parametros = m_p + (m_com_constante ? 1 : 0);
145       if(observacoes <= parametros)
146          return(false);
147 
148       CMatrixDouble X(observacoes, parametros);
149       double dependente[];
150       ArrayResize(dependente, observacoes);
151 
152       for(int t = m_p, linha = 0; t < tamanho; t++, linha++)
153         {
154          dependente[linha] = m_serie_diferenciada[t];
155          int coluna = 0;
156          if(m_com_constante)
157             X.Set(linha, coluna++, 1.0);
158          for(int i = 1; i <= m_p; i++)
159             X.Set(linha, coluna++, m_serie_diferenciada[t - i]);
160         }
161 
162       double coeficientes[], erros_padrao[], soma_quadrados = 0.0;
163       if(!MQO(X, dependente, observacoes, parametros, coeficientes, erros_padrao,
      soma_quadrados, false))
164          return(false);
165 
```

```
166       int coluna = 0;
167       m_constante = m_com_constante ? coeficientes[coluna++] : 0.0;
168       for(int i = 0; i < m_p; i++)
169          m_phi[i] = coeficientes[coluna++];
170       return(true);
171      }
```

Lendo o código

Linha 135: sem MA ($q = 0$), entra no atalho.
Linhas 137–141: sem MA e sem AR ($p = q = 0$), o modelo é só $z_t = c + \varepsilon_t$, e o melhor c é a média dos retornos da janela.
Linhas 152–160: monta a tabela da seção 2: uma linha por dia a partir do p-ésimo, coluna de 1 (se houver constante) e os p retornos anteriores.
Linha 163: o MQO.
Linhas 166–169: lê os coeficientes na ordem em que as colunas foram montadas: primeiro a constante, depois os ϕ.

Ou seja, para um ARIMA sem MA, o "Hannan-Rissanen" do código é só um MQO. O nome da função cobre os dois casos.

8 Estágio 1 no código: o AR comprido

8.1 Quantos dias para trás

tg/codigo/Arima.mqh

```
173    int ordem_longa = (int)MathFloor(MathPow(MathLog((double)tamanho), 2.0));
174    ordem_longa = MathMax(ordem_longa, m_p + m_q + 1);
175    ordem_longa = MathMin(ordem_longa, tamanho / 5);
176
177    if(ordem_longa < 1 || tamanho - ordem_longa < m_p + m_q + 10)
178       return(false);
179
180    double coeficientes_ar_longo[], erros_estimados[];
181    if(!EstimarAR(m_serie_diferenciada, ordem_longa, coeficientes_ar_longo, erros_estimados))
182       return(false);
```

Lendo o código

Linha 173: a ordem do AR comprido é $m = \lfloor (\ln N)^2 \rfloor$. Na janela do robô, $N = 239$, $\ln 239 = 5{,}476$ e $(\ln 239)^2 = 29{,}99$. Arredondando para baixo, $m = 29$.
Linha 174: m nunca é menor que $p + q + 1$.
Linha 175: nem maior que $N/5$, para sobrarem linhas suficientes na regressão.
Linhas 177–178: desiste se a série for curta demais.
Linha 181: chama o AR comprido, que devolve os erros estimados em `erros_estimados`.

A regra $\lfloor \log^2 N \rfloor$ é a de Gómez e Maravall (1998, p. 28), apoiados em Hannan e Rissanen (1982, p. 88), que pedem que m cresça com o tamanho da série, mas devagar. A seção 16.1 discute se ela é boa numa janela de 239 dias.

8.2 O AR comprido

tg/codigo/Arima.mqh

```
93  bool CArima::EstimarAR(const double &z[], const int ordem, double &coeficientes[], double &
        residuos[])
94    {
95     int tamanho = ArraySize(z);
96     int observacoes = tamanho - ordem;
97     int parametros = ordem + 1;
98
99     if(ordem <= 0 || observacoes <= parametros)
100       return(false);
101
102    CMatrixDouble X(observacoes, parametros);
103    double dependente[];
104    ArrayResize(dependente, observacoes);
105
106    for(int t = ordem, linha = 0; t < tamanho; t++, linha++)
107      {
108       dependente[linha] = z[t];
109       X.Set(linha, 0, 1.0);
110       for(int i = 1; i <= ordem; i++)
111          X.Set(linha, i, z[t - i]);
112      }
113
114    double erros_padrao[], soma_quadrados = 0.0;
115    if(!MQO(X, dependente, observacoes, parametros, coeficientes, erros_padrao, soma_quadrados,
         false))
116       return(false);
117
118    ArrayResize(residuos, tamanho);
119    ArrayInitialize(residuos, 0.0);
120    for(int t = ordem; t < tamanho; t++)
121      {
122       double ajuste = coeficientes[0];
123       for(int i = 1; i <= ordem; i++)
124          ajuste += coeficientes[i] * z[t - i];
125       residuos[t] = z[t] - ajuste;
126      }
127
128    return(true);
129  }
```

Lendo o código

Linhas 96–97: a tabela tem $N - m$ linhas (os primeiros m dias não têm m dias anteriores) e $m + 1$ colunas. No pregão real, 210 linhas e 30 colunas.
Linhas 106–112: a mesma tabela da seção 2, com m colunas de retornos. A coluna 0 é sempre a constante, mesmo que o modelo final não tenha constante.
Linha 115: o MQO.
Linhas 118–126: o que interessa. Para cada dia, o resíduo é o retorno menos o que o AR comprido previa (linha 125). É o $\hat{\varepsilon}_t$ da seção 5.
Linha 119: os primeiros m dias ficam com resíduo zero, porque não tiveram previsão. Isso vai importar no estágio 2.

8.3 O que o estágio 1 produziu no pregão real

A Figura 3 mostra o AR(29) do pregão de 13/05/2021. As barras laranja são os 29 coeficientes estimados. Os pontos azuis são os pesos que o AR equivalente ao MA(1) teria, com o θ que o robô estimou no final (seção 4). O primeiro coeficiente bate com o peso teórico. Os outros 28 são, em boa parte, ruído: com 210 linhas, cada coeficiente a mais é estimado com erro. Não faz mal, porque o estágio 1 não usa os coeficientes, só os resíduos.

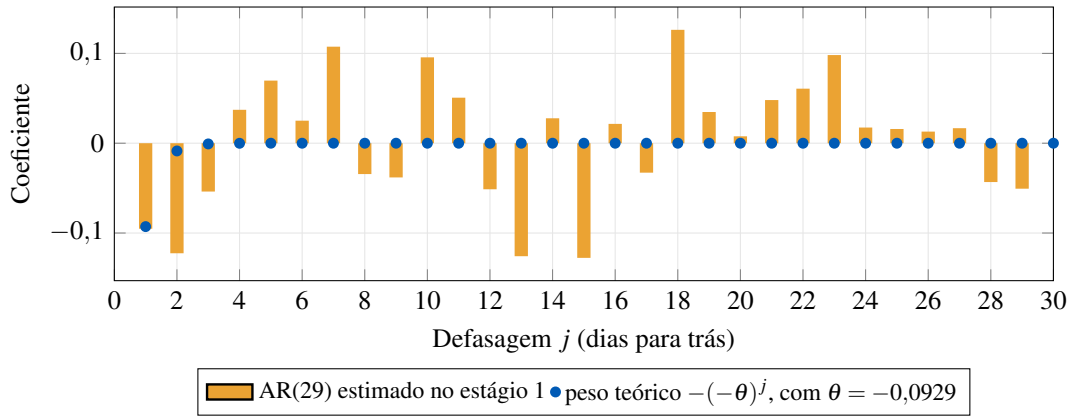


Figura 3: Pregão de 13/05/2021: os coeficientes do AR comprido e os pesos do AR infinito equivalente ao MA(1) estimado.

Na série simulada, onde o erro verdadeiro é conhecido, dá para conferir a qualidade dos erros estimados (Figura 4): a correlação entre o erro estimado e o verdadeiro foi de 0,922.

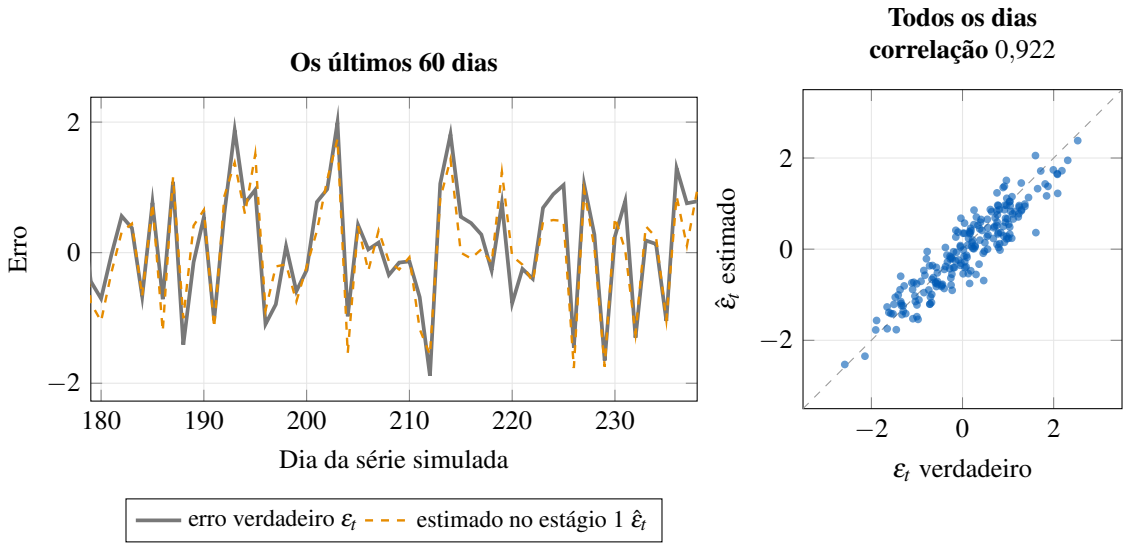


Figura 4: Série simulada: os erros estimados pelo AR comprido seguem os verdadeiros de perto. Não são iguais, e é por isso que o resultado do Hannan-Rissanen é uma aproximação.

9 Estágio 2 no código: a regressão que dá c e θ

tg/codigo/Arima.mqh

```
184    int primeiro = MathMax(ordem_longa + m_q, m_p);
185    int observacoes = tamanho - primeiro;
186    int parametros = m_p + m_q + (m_com_constante ? 1 : 0);
187
188    if(observacoes <= parametros + 5)
189       return(false);
190
191    CMatrixDouble X(observacoes, parametros);
192    double dependente[];
193    ArrayResize(dependente, observacoes);
194
195    for(int t = primeiro, linha = 0; t < tamanho; t++, linha++)
```

```
196       {
197        dependente[linha] = m_serie_diferenciada[t];
198        int coluna = 0;
199        if(m_com_constante)
200           X.Set(linha, coluna++, 1.0);
201        for(int i = 1; i <= m_p; i++)
202           X.Set(linha, coluna++, m_serie_diferenciada[t - i]);
203        for(int j = 1; j <= m_q; j++)
204           X.Set(linha, coluna++, erros_estimados[t - j]);
205       }
206
207    double coeficientes[], erros_padrao[], soma_quadrados = 0.0;
208    if(!MQO(X, dependente, observacoes, parametros, coeficientes, erros_padrao, soma_quadrados,
         false))
209       return(false);
210
211    int coluna = 0;
212    m_constante = m_com_constante ? coeficientes[coluna++] : 0.0;
213    for(int i = 0; i < m_p; i++)
214       m_phi[i] = coeficientes[coluna++];
215    for(int j = 0; j < m_q; j++)
216       m_theta[j] = coeficientes[coluna++];
217    return(true);
```

Lendo o código

Linha 184: o primeiro dia usado é $\max(m+q,\ p)$. O motivo é a linha 119: os erros estimados só existem do dia m em diante, e a linha precisa de $\hat{\varepsilon}_{t-q}$. No pregão real, $m = 29$ e $q = 1$, então a regressão começa no dia 30 e tem 209 linhas.
Linha 186: as colunas são a constante (se houver), os p retornos e os q erros: $1 + p + q$.
Linhas 195–205: a tabela. As linhas 201–202 são as colunas da parte AR, iguais às do atalho. As linhas 203–204 são a novidade: colunas com os erros estimados no estágio 1, deslocados j dias.
Linha 208: o MQO.
Linhas 211–216: lê os coeficientes na ordem das colunas: constante, $\phi_1 \dots \phi_p$, $\theta_1 \dots \theta_q$.

No pregão real, o modelo era um ARIMA(0,1,1) com constante, então a tabela tem só duas colunas:

dia t	z_t	1	$\hat{\varepsilon}_{t-1}$
08/07/2020	1,872	1	−0,582
09/07/2020	−2,277	1	1,533
10/07/2020	1,657	1	−2,351
13/07/2020	−1,567	1	1,630
⋮	(209 linhas)		

Com uma coluna só além da constante, a regressão é uma reta num gráfico de dispersão: cada ponto é um dia, com o erro estimado de ontem no eixo horizontal e o retorno de hoje no vertical (Figura 5b, na seção 13). A inclinação da reta é o θ e a altura no zero é o c. Deu $c = 0{,}0524\%$ e $\theta = -0{,}0929$.

10 Depois dos dois estágios: os resíduos finais

Um detalhe que costuma passar despercebido: os $\hat{\varepsilon}_t$ do estágio 1 **não** são os resíduos que o robô usa depois. Eles vieram do AR comprido, que é outro modelo. Com os coeficientes finais em mãos, o código recalcula os erros do zero, um dia de cada vez, com o próprio ARIMA:

tg/codigo/Arima.mqh

```
220 void CArima::CalcularResiduos(void)
```

```
221   {
222    int tamanho = ArraySize(m_serie_diferenciada);
223    ArrayResize(m_residuos, tamanho);
224    ArrayInitialize(m_residuos, 0.0);
225
226    m_inicio_recursao = MathMax(m_p, m_q);
227    m_inicio_verossimilhanca = MathMax(m_inicio_recursao, m_aquecimento_minimo);
228
229    for(int t = m_inicio_recursao; t < tamanho; t++)
230      {
231       double ajuste = m_constante;
232       for(int i = 0; i < m_p; i++)
233          ajuste += m_phi[i] * m_serie_diferenciada[t - 1 - i];
234       for(int j = 0; j < m_q; j++)
235          ajuste += m_theta[j] * m_residuos[t - 1 - j];
236       m_residuos[t] = m_serie_diferenciada[t] - ajuste;
237      }
238   }
```

Lendo o código

Linha 226: começa no dia $\max(p,q)$. Antes dele, os erros ficam em zero (linha 224).
Linhas 231–235: o que o modelo previa para o dia t: constante, mais a parte AR com os retornos anteriores, mais a parte MA com os *resíduos já calculados* dos dias anteriores (linha 235).
Linha 236: o resíduo do dia é o retorno menos essa previsão.

É uma conta recursiva: o resíduo de hoje usa o de ontem, que usou o de anteontem. Dá para seguir à mão. No pregão real ($c = 0{,}0524\%$, $\theta = -0{,}0929$), a conta é $e_t = z_t - (c + \theta\, e_{t-1})$:

dia	retorno z_t	resíduo de ontem e_{t-1}	previsto $c+\theta e_{t-1}$	resíduo e_t
27/05/2020	1,313	0,000	0,052	1,261
28/05/2020	−0,806	1,261	−0,065	−0,741
29/05/2020	2,842	−0,741	0,121	2,721
		⋮		
10/05/2021	1,304	3,522	−0,275	1,579
11/05/2021	1,805	1,579	−0,094	1,900
12/05/2021	−1,482	1,900	−0,124	−1,358

Valores em % ao dia. Na primeira linha, o resíduo de ontem é o zero do início da recursão (linha 224).

Para conferir uma linha: multiplique o resíduo de ontem por θ e some c, o que dá o previsto; o retorno menos o previsto dá o resíduo, que vira o "resíduo de ontem" da linha seguinte. Diferenças na última casa vêm do arredondamento da tabela.

Esses e_t vão para três lugares: a previsão de amanhã (seção 11), a verossimilhança que o AIC compara na escolha das ordens (seção 12) e o GARCH, que estima a volatilidade em cima deles. Na série simulada, os resíduos finais tiveram correlação de 0,995 com o erro verdadeiro, mais que os do estágio 1 (0,922): com o modelo certo e poucos coeficientes, a conta recursiva acerta mais que o AR comprido, que carrega o ruído de 29 coeficientes.

11 A previsão de amanhã

tg/codigo/Arima.mqh

```
310 double CArima::PreverDiferenciado(void)
311   {
```

```
312    if(!m_ajustado)
313       return(0.0);
314
315    int tamanho = ArraySize(m_serie_diferenciada);
316    double previsao = m_constante;
317
318    for(int i = 0; i < m_p; i++)
319      {
320       int indice = tamanho - 1 - i;
321       if(indice >= 0)
322          previsao += m_phi[i] * m_serie_diferenciada[indice];
323      }
324    for(int j = 0; j < m_q; j++)
325      {
326       int indice = tamanho - 1 - j;
327       if(indice >= 0)
328          previsao += m_theta[j] * m_residuos[indice];
329      }
330
331    return(previsao);
332   }
```

Lendo o código

Linha 316: começa pela constante.
Linhas 318–323: soma ϕ_i vezes os últimos retornos. O índice `tamanho - 1` é o último dia da janela.
Linhas 324–329: soma θ_j vezes os últimos resíduos finais.
O erro de amanhã não aparece: ele é desconhecido, e o melhor palpite para uma surpresa é zero.

No pregão real, o último resíduo da janela foi $e_N = -1{,}358\%$ (a última linha da tabela anterior), e

$$\hat{z}_{N+1} = c + \theta\, e_N = 0{,}0524\% + (-0{,}0929) \times (-1{,}358\%) = 0{,}0524\% + 0{,}1261\% = 0{,}1785\%.$$

Positiva, então o ARIMA deu sinal de compra. O EA usa esse número direto quando $d = 1$:

tg/codigo/ARIMA_GARCH_EA.mq5

```
143 double PrevisaoDoRetorno(const double &serie[])
144   {
145    if(g_arima.D() == 1)
146       return(g_arima.PreverDiferenciado());
147    return(g_arima.PreverNivel() - serie[ArraySize(serie) - 1]);
148   }
```

E entrega os resíduos ao GARCH, sem os dias de aquecimento:

tg/codigo/ARIMA_GARCH_EA.mq5

```
150 bool EstimarVolatilidade(double &volatilidade, double &limiar)
151   {
152    double residuos[];
153    if(!g_arima.ObterResiduos(residuos))
154       return(false);
155    if(!g_garch.Ajustar(residuos))
```

tg/codigo/Arima.mqh

```
357 bool CArima::ObterResiduos(double &saida[]) const
358   {
359    if(!m_ajustado)
360       return(false);
361
```

```
362    int validos = ArraySize(m_residuos) - m_inicio_recursao;
363    if(validos <= 0)
364       return(false);
365
366    ArrayResize(saida, validos);
367    for(int i = 0; i < validos; i++)
368       saida[i] = m_residuos[m_inicio_recursao + i];
369
370    return(true);
371   }
```

A linha 362 descarta os primeiros $\max(p,q)$ resíduos, os zeros do início da recursão. É a correção do bug 5 da versão anterior do EA, em que esses zeros iam para o GARCH e contaminavam a variância.

12 Na escolha das ordens: 25 modelos de uma vez

A cada 21 pregões, o EA reescolhe p e q:

tg/codigo/ARIMA_GARCH_EA.mq5

```
134 void ReselecionarOrdensSeVencidas()
135   {
136    bool vencidas = !g_ordens_definidas ||
137                    (Barras_Entre_Reselecao > 0 && g_barras_desde_selecao >=
       Barras_Entre_Reselecao);
138    if(vencidas && SelecionarOrdens())
139       g_arima.Definir(g_ordem_p, g_ordem_d, g_ordem_q, IncluirConstante);
140    g_barras_desde_selecao++;
141   }
```

A escolha testa todas as combinações de p e q de 0 a 4 e fica com o menor AIC:

tg/codigo/SelecaoModelo.mqh (Apêndice B)

```
51    int aquecimento_comum = MathMax(p_maximo, q_maximo);
52    double melhor_criterio = DBL_MAX;
53
54    for(int p = 0; p <= p_maximo; p++)
55       for(int q = 0; q <= q_maximo; q++)
56         {
57          if(p == 0 && q == 0 && !com_constante && d != 0)
58             continue;
59
60          CArima candidato;
61          candidato.Definir(p, d, q, com_constante);
62          candidato.DefinirAquecimentoMinimo(aquecimento_comum);
63
64          if(!candidato.Ajustar(serie))
65             continue;
66
67          double aic = candidato.AIC();
68          if(!MathIsValidNumber(aic))
69             continue;
70
71          if(aic < melhor_criterio)
72            {
73             melhor_criterio = aic;
74             resultado.p = p;
75             resultado.q = q;
76             resultado.valido = true;
77            }
78         }
```

Lendo o código

Linhas 54–55: os dois laços percorrem os 25 pares (p,q).
Linhas 60–64: para cada par, um ARIMA novo é criado e ajustado. O `Ajustar` da linha 64 é o mesmo da seção 6, então o Hannan-Rissanen roda inteiro em cada candidato. Os 20 pares com $q \geq 1$ passam pelos dois estágios; os 5 com $q = 0$, pelo atalho.
Linha 62 (e 51): todos os candidatos medem a verossimilhança sobre os mesmos dias, descartando os 4 primeiros. Sem isso, o modelo menor somaria mais dias e ganharia por construção.
Linhas 67–77: guarda o par de menor AIC.

O AIC vem dos resíduos finais da seção 10:

tg/codigo/Arima.mqh

```
252    double soma_quadrados = 0.0;
253    for(int t = m_inicio_verossimilhanca; t < tamanho; t++)
254       soma_quadrados += m_residuos[t] * m_residuos[t];
255
256    m_sigma2 = soma_quadrados / (double)m_observacoes_efetivas;
257    if(m_sigma2 <= 0.0 || !MathIsValidNumber(m_sigma2))
258      {
259       m_log_verossimilhanca = -DBL_MAX;
260       return;
261      }
262
263    m_log_verossimilhanca = -0.5 * (double)m_observacoes_efetivas *
264                            (MathLog(2.0 * M_PI) + MathLog(m_sigma2) + 1.0);
```

A soma dos resíduos ao quadrado (linhas 253–254) dá a variância σ^2 (linha 256), e a log-verossimilhança de uma normal com essa variância é a fórmula das linhas 263–264. O AIC é $-2\log L + 2k$, com k parâmetros. Ou seja: o Hannan-Rissanen não escolhe as ordens, mas é ele que produz os resíduos que decidem a escolha. Se ele estimar mal um modelo com MA, os resíduos desse modelo pioram, e a escolha tende a ficar com um modelo sem MA.

13 Um pregão real, inteiro: 13/05/2021

Juntando as seções anteriores num dia só. Este é o pregão de 13/05/2021 da configuração ARIMA-GARCH Sharpe, no traço da refação de 05/10/2026. O modelo em uso era um **ARIMA(0,1,1) com constante**: nenhum retorno passado, um erro passado. É o caso mais simples em que os dois estágios rodam.

1. **Dados** (`CarregarSerie`, `Diferenciar`). Os 240 fechamentos antes do pregão, de 25/05/2020 a 12/05/2021 (último: R$ 24,78). O log de cada um e a diferença: $N = 239$ log-retornos.
2. **Estágio 1** (linhas 173–182 e 93–129). AR(29) com constante, por MQO, em 210 linhas. O desvio-padrão dos resíduos foi de 2,91% ao dia.
3. **Estágio 2** (linhas 184–217). $z_t = c + \theta\,\hat{\varepsilon}_{t-1}$, em 209 linhas: $c = 0{,}0524\%$ e $\theta = -0{,}0929$.
4. **Resíduos finais** (linhas 220–238). Recalculados com c e θ. O do último dia: $e_N = -1{,}358\%$.
5. **Previsão** (linhas 310–332). $\hat{z}_{N+1} = 0{,}1785\%$, sinal de compra.

O que esse dia ensina. O θ é pequeno e a nuvem é quase redonda. Ainda assim, nesse pregão a parte MA pesou *mais* que a constante na previsão (0,1261% contra 0,0524%), porque o erro de ontem foi grande ($-1{,}358\%$) e o θ negativo transforma um erro negativo em empurrão para cima. É a leitura de "correção": depois de um dia pior que o esperado, o modelo espera uma pequena compensação. A máxima

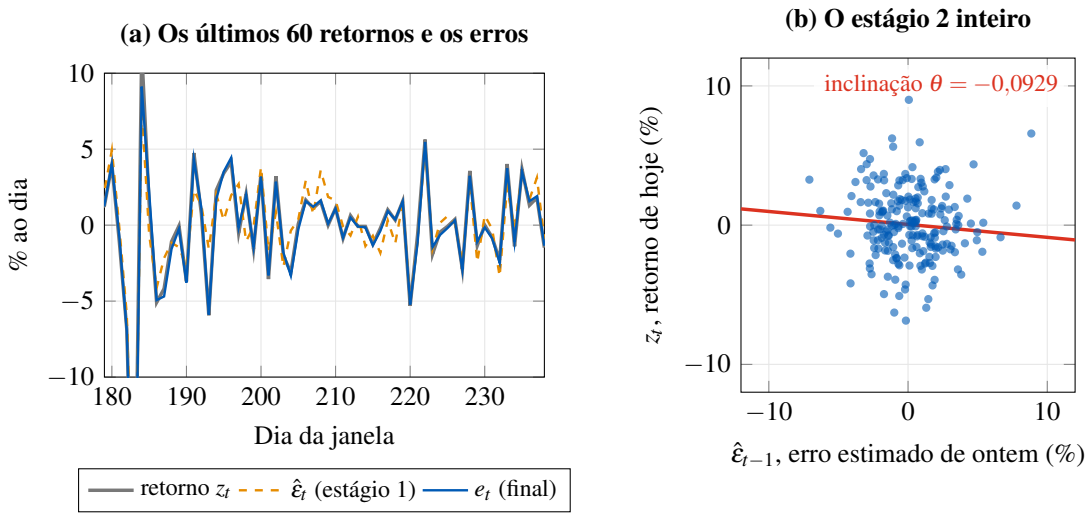


Figura 5: O pregão de 13/05/2021, refeito com as mesmas regras do robô. Em (b), cada ponto é uma linha da tabela do estágio 2. A nuvem é quase redonda e a reta, quase deitada: o erro de ontem diz pouco sobre o retorno de hoje. Em (a), um dia de queda forte sai cortado na escala.

verossimilhança, na mesma janela, daria $\theta = -0{,}1248$ e previsão de 0,2646%: números diferentes, mesmo sinal, mesma decisão.

Conferência

Os valores de c, θ e da previsão refeitos aqui são **idênticos**, em todas as casas gravadas, aos que o robô registrou no traço desse pregão (`refeito_2026-10-05/traco_demo/traco_demo_GARCH.csv`). O mesmo vale para o pregão de 05/06/2020, com um ARIMA(0,1,4) e quatro θ. O relatório descreve o que o robô de fato fez.

14 Quanto ele trabalha no robô

Todo pregão, na estimação do modelo em uso. Os dois estágios só rodam se o modelo tiver termo MA. No traço de jul/2019 a jun/2021, isso aconteceu em 105 dos 496 pregões (21%), e o atalho rodou nos outros 391 (Figura 6).

A cada 21 pregões, na escolha das ordens, 20 dos 25 candidatos passam pelos dois estágios. É aqui que o método mais trabalha: ele decide, indiretamente, se o robô vai usar um modelo com MA ou não.

Parte III

Quão bem ele estima

15 Mil simulações

Uma série só não diz se o método é bom. Para isso, geraram-se mil séries com coeficientes conhecidos, cada uma do tamanho da janela do robô (239 retornos), e cada uma foi estimada de três jeitos: Hannan-Rissanen (o do robô), máxima verossimilhança (a opção experimental `MetodoARIMA=1`, que parte do Hannan-Rissanen e refina) e um AR(1) que ignora o MA.

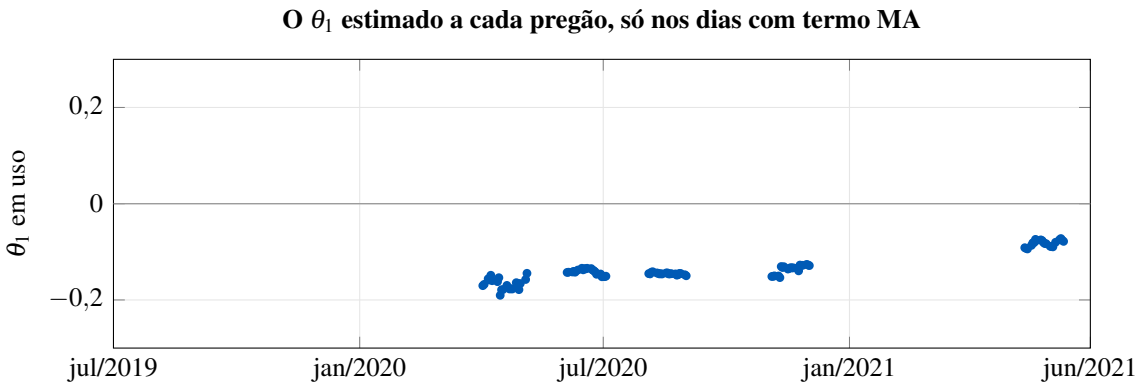


Figura 6: Traço da configuração ARIMA-GARCH Sharpe. Cada ponto é um pregão em que o modelo em uso tinha termo MA e os dois estágios rodaram. Os blocos correspondem aos períodos entre duas escolhas de ordem. Nos dias sem ponto o modelo não tinha MA.

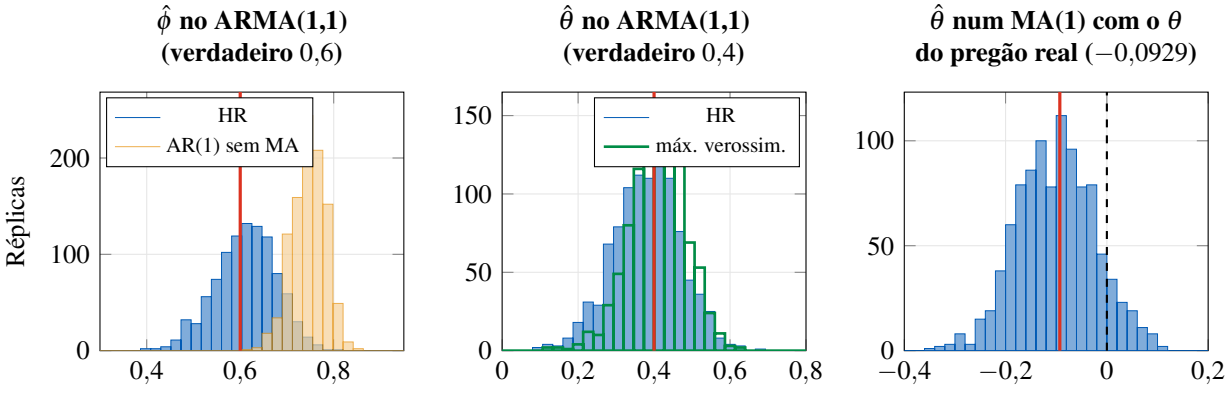


Figura 7: Distribuição das estimativas em 1000 séries simuladas. A linha vermelha é o valor verdadeiro; a tracejada preta, no gráfico da direita, é o zero.

	ARMA(1,1), $\phi = 0{,}6$, $\theta = 0{,}4$		MA(1), $\theta = -0{,}0929$
Método	$\hat{\phi}$ média (desvio)	$\hat{\theta}$ média (desvio)	$\hat{\theta}$ média (desvio)
Hannan-Rissanen (robô)	0,612 (0,065)	0,380 (0,089)	−0,100 (0,077)
Máxima verossimilhança	0,584 (0,066)	0,410 (0,074)	−0,099 (0,069)
AR(1) ignorando o MA	0,746 (0,036)	–	–

Ignorar o MA distorce. O AR(1) sem MA erra o ϕ de forma sistemática (média 0,746 para um verdadeiro de 0,6). É a justificativa para ter o termo MA e, portanto, um método que o estime.

O HR chega perto, com um pouco mais de erro que a máxima verossimilhança. No θ, o desvio do HR foi 0,089 e o da máxima verossimilhança, 0,074, e a média do HR ficou um pouco abaixo do verdadeiro. É o comportamento descrito na literatura: Gómez e Maravall (1998, p. 29) dizem que os estimadores do segundo estágio são consistentes, mas têm viés e não são os mais eficientes.

No tamanho dos coeficientes do robô, o erro de estimação é do tamanho do coeficiente. Com $\theta = -0{,}0929$, o desvio da estimativa foi 0,077, quase igual ao próprio θ. Em 90% das séries o HR acertou o sinal, e nas demais estimou um θ com o sinal trocado. Isso não é defeito do método: com 239 retornos, um efeito desse tamanho mal se distingue do zero, e a máxima verossimilhança tem o mesmo problema (0,069).

16 O seu código comparado ao método da literatura

O código segue a ideia de Hannan e Rissanen, com algumas escolhas próprias. Comparando com a descrição de Gómez e Maravall (1998, p. 28–29):

Ponto	Gómez e Maravall (1998)	O seu código (`Arima.mqh`)
Estágio 1 só com MA	sim	sim (linha 135: com $q = 0$, MQO direto)
Como estima o AR longo	autocorrelações e recursão de Durbin-Levinson	MQO com constante (linhas 93–129)
Ordem do AR longo	$\max(\lfloor \log^2(N) \rfloor,\ 2\max(p,q))$	$\max(\lfloor \ln^2 N \rfloor,\ p+q+1)$, no máximo $N/5$ (linhas 173–175)
Terceiro passo (correção do viés)	sim, quando há MA	**não**
Critério para escolher (p,q)	BIC	AIC sobre amostra comum
Invertibilidade ($\lvert\theta\rvert < 1$)	–	não é verificada

- **Sem o terceiro passo**, o robô usa os coeficientes do estágio 2 como estão. É de onde vem o pequeno viés do θ nas simulações. A máxima verossimilhança faria o papel desse refinamento; foi testada como opção em 15/10/2026 e não adotada.
- **MQO no lugar de Durbin-Levinson** dá coeficientes parecidos, mas não iguais, para o AR comprido. Os dois são formas aceitas de estimar um AR.
- **A invertibilidade não é imposta.** A seção 4 mostrou por que ela importa: com $\lvert\theta\rvert \geq 1$ os pesos não encolhem. Nos dias do traço os θ ficaram pequenos (Figura 6), longe de 1, então isso não pesou ali.

16.1 Um ponto para pensar: a ordem do AR comprido numa janela pequena

A regra $\ln^2 N$ vem de resultados para amostras grandes. Com $N = 239$ ela dá 29 defasagens, estimadas com 210 linhas. Na série simulada, refazendo tudo com cada ordem de 1 a 47, a qualidade dos erros

estimados caiu à medida que a ordem cresceu (Figura 8). Com poucas defasagens o AR comprido já capta um ARMA(1,1), e cada defasagem a mais acrescenta um coeficiente estimado com ruído. É o mesmo efeito visto na Figura 3.

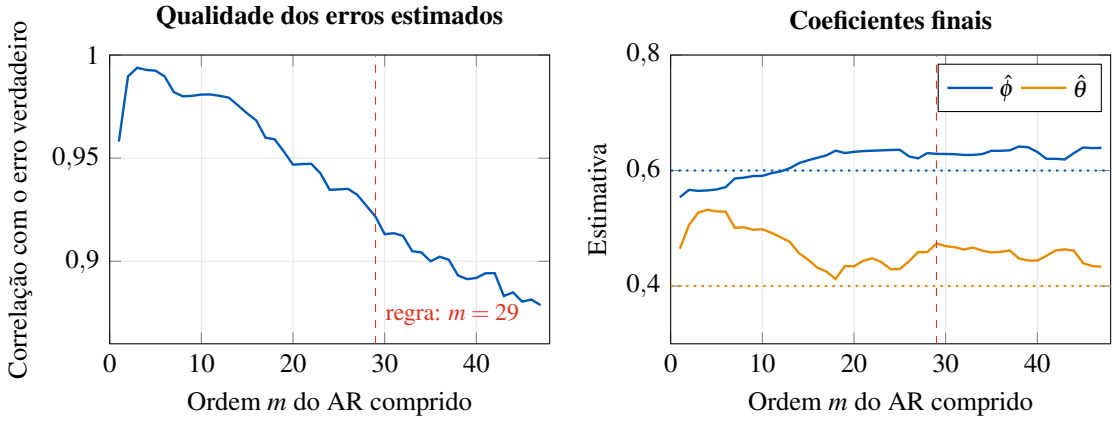


Figura 8: Uma única série simulada (ARMA(1,1), $\phi = 0{,}6$, $\theta = 0{,}4$), refeita com cada ordem do AR comprido. As linhas pontilhadas são os valores verdadeiros. A correlação foi medida sobre os mesmos dias em todas as ordens.

Até onde isso vale

É uma série só, com coeficientes bem maiores que os da PETR4. Não se testou se uma ordem menor mudaria alguma decisão do robô. Fica registrado como ponto em aberto (`ia/INCERTEZAS.md`, item 84), não como defeito.

17 Perguntas que a banca pode fazer

Se a banca perguntar

O que é o Hannan-Rissanen, em uma frase?
Um jeito de estimar um modelo ARMA com duas regressões de mínimos quadrados: a primeira, um modelo autorregressivo longo, estima os erros que não se observam; a segunda usa esses erros estimados como variável para obter os coeficientes.

Por que um AR longo consegue estimar os erros de um modelo com MA?
Porque, se o MA é invertível, o erro de hoje pode ser escrito como uma soma ponderada dos retornos passados, com pesos que diminuem (seção 4). Um AR longo aproxima essa soma.

Por que Hannan-Rissanen e não máxima verossimilhança?
Porque o modelo é reestimado todo pregão e, na escolha das ordens, 25 vezes de uma vez. O Hannan-Rissanen são duas regressões lineares: não precisa de ponto de partida, de taxa de aprendizado nem de número de iterações, não depende da escala dos preços e não tem problema de convergência. A literatura o usa justamente na identificação automática de modelos (Gómez e Maravall, 1998, p. 10). O custo é uma estimativa um pouco menos precisa: nas simulações, o desvio do θ foi 0,089, contra 0,074 da máxima verossimilhança.

Onde isso está no código?
Na função `CArima::HannanRissanen`, no Apêndice B. As linhas 173–182 são o estágio 1, as linhas 184–217 o estágio 2. Antes disso, as linhas 135–171 tratam o caso sem termo MA, que é só uma regressão.

O "MA" do ARIMA é a média móvel da análise técnica?
Não. O MA do ARIMA é uma combinação dos *erros* passados, não dos preços. A ponte com a média móvel de Almeida (2016) é pela parte AR, não pela parte MA.

Como vocês sabem que a implementação está certa?
Duas conferências. Em série simulada com coeficientes conhecidos, o método recupera os valores verdadeiros dentro do erro esperado. E a estimação de pregões reais, refeita fora do robô com os mesmos passos, dá os mesmos coeficientes e a mesma previsão que o robô registrou.

Glossário rápido

MQO	Mínimos quadrados ordinários: escolhe os coeficientes de uma regressão que tornam mínima a soma dos erros ao quadrado. Conta fechada, sem iteração.
Regressão	Explicar uma coluna de números (y) por uma combinação de outras colunas (x).
AR(p)	Modelo autorregressivo: o valor de hoje depende dos p valores anteriores.
MA(q)	Médias móveis (de erros): o valor de hoje depende dos q erros anteriores.
Erro, surpresa	A parte do valor de hoje que o modelo não consegue prever.
Resíduo	A diferença entre o valor observado e o que o modelo ajustado previa. É a estimativa do erro.
Invertível	Um MA cujos erros podem ser reescritos como soma de valores passados ($\lvert\theta\rvert < 1$ num termo).
Recursão	Conta em que o resultado de cada passo usa o resultado do passo anterior.
Viés	Tendência de uma estimativa a errar sempre para o mesmo lado.

Monte Carlo Repetir um experimento muitas vezes com dados simulados, para ver como um método se comporta em média.

Procedência. Dados gerados por `ferramentas/HannanRissanenDetalhado.mq5` (rodado por `ferramentas/scripts/hannan_rissanen.sh`) em `dados/resultados/hannan_rissanen_2026-10-30/`; as contas à mão (tabela da recursão, $\ln N$, pesos π_j) por `ferramentas/scripts/hannan_rissanen_exemplo.sh`, que só lê aqueles arquivos; o traço da refação de 05/10/2026 resumido em `dados/resultados/robo_em_acao_2026-10-30/`. Código lido de `tg/codigo/` (a versão dos Apêndices A e B, mesma numeração de linhas) e de `robos/Include/Numerico.mqh`. Semente das simulações: 20261030. Obras citadas: Gómez e Maravall (1998), em `bibliografia/pdfs/Gomez-1998.pdf`; Hannan e Rissanen (1982) é citado através deles (o artigo original não foi lido). Material de estudo, não faz parte do TG.